JADE™ GLOBAL

React - Basics

React Architecture
JSX

©2025 Jade Global All rights reserved

What We'll Cover

Your complete roadmap for the React.js Beginner Module

01 Foundations

History, types, and how React thinks

- Intro to React
- Why React?
- SPA Concept
- Setup & Vite

02 Core Concepts

Building blocks of every React app

- Components
- JSX Syntax
- Props
- State & Hooks

03 Interactivity

Making apps dynamic and interactive

- Event Handling
- Conditional Render
- Lists & Keys
- Basic Forms

04 Advanced Basics

Fetch data, style, and navigate

- Styling
- useEffect
- API Fetch
- React Router

05 Ship It!

Debug, build, and go live!

- Debugging
- Build & Deploy
- Mini Project
- Common Mistakes

01 Introduction to React

What is React and where did it come from?

What is React?

React is a JavaScript library for building user interfaces (UIs). Created by Facebook (Meta) in 2013, React lets you build fast, interactive web apps from small, reusable pieces called **components**.

🔑 **Simple Analogy:** Think of React like LEGO bricks. Each brick (component) is a small, self-contained piece. You snap bricks together to build a complete structure — your web app.

React is NOT a full framework — it handles only the View layer (what the user sees). You pair it with other tools for routing, state, and API calls.

Created by Meta

Originally built for Facebook's newsfeed in 2011, open-sourced in 2013

Most Popular UI Library

Used by Netflix, Airbnb, Twitter, Instagram & millions of developers

Virtual DOM

React uses a Virtual DOM to update only changed parts — making it blazing fast

02 Why React?

Problems React solves — and why developers love it

JADE GLOBAL

BEFORE React (Vanilla JS)

- ❌ Manual DOM manipulation — messy & slow
- ❌ Hard to keep UI in sync with data
- ❌ No reusable components — copy-pasting HTML
- ❌ Spaghetti code as app grows
- ❌ No built-in state management

→

WITH React

- ✅ Virtual DOM — React updates only what changed
- ✅ Declarative UI — describe WHAT, not HOW
- ✅ Reusable components — build once, use anywhere
- ✅ Unidirectional data flow — predictable & easy to debug
- ✅ Huge ecosystem — React Router, Redux, Next.js...

03 Single Page Applications (SPA)

How SPAs work vs traditional multi-page websites

Traditional Website (MPA)

1. User clicks a link
2. Browser sends request to server
3. Server returns a WHOLE NEW HTML page
4. Browser re-renders everything
5. Page flickers / feels slow

Result: Slow, full page reloads on every navigation

React SPA

1. Browser loads ONE HTML file (index.html)
2. React JS bundle loads
3. User clicks a link
4. React swaps only the changed components
5. No page reload — instant, app-like experience

Result: Blazing fast, feels like a native mobile app

04 React Setup with Vite

Getting your development environment up and running

STEP 01 Install Node.js

```
Download from nodejs.org
Choose LTS version (v18+)
Verify: node -v && npm -v
```

STEP 02 Create React App

```
npm create vite@latest my-app
-- --template react
cd my-app
```

STEP 03 Install & Run

```
npm install
npm run dev
# Opens on localhost:5173
```

Generated Project Structure

DIR

```
my-app/
├── src/
│   ├── App.jsx        ← Main component
│   ├── main.jsx       ← Entry point
│   └── App.css
├── index.html         ← Single HTML file
└── vite.config.js
```

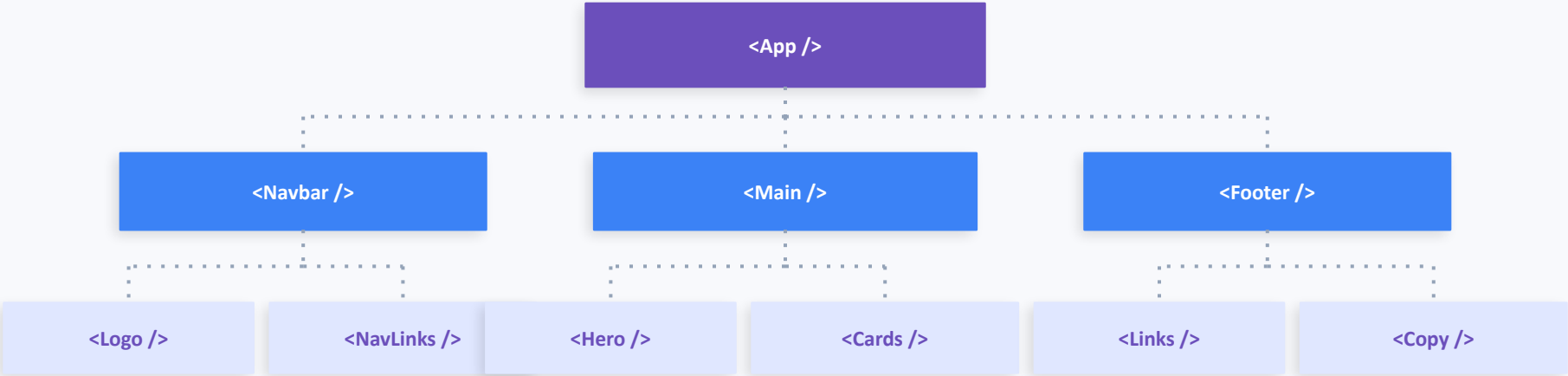
Key Files Explained

`main.jsx`	Entry point — mounts your App into index.html
`App.jsx`	Root component — your first component to edit
`index.html`	The ONE HTML file — React injects everything here

05 React Architecture

Component-based architecture — UI as a tree of components

Every React App is a Tree of Components



💡 **Key Insight:** Each box = one component (a reusable piece of UI). A component can hold other components inside it. Data flows from parent ⟶ child (top-down). This predictable flow makes React easy to reason about.

06 Components

Functional components — the building blocks of React

JADE GLOBAL

What is a Component?

A component is a JavaScript function that returns JSX (HTML-like code).

Rules to remember:

- Name starts with a Capital letter (Button, not button)
- Must return JSX
- Can be used like an HTML tag: <Button />

🔁 **Reusability:** Write once, use many times with different data.

✏️ **Mini Exercise:**

Create a <Card /> component that shows a name and a job title. Use it 3 times in <App />.

Code Example

JSX

```jsx
// Simple functional component
function Greeting() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <p>Welcome to React!</p>
    </div>
  );
}

// Use it like an HTML tag:
// <Greeting />
```

Component Composition

JSX

```jsx
function App() {
  return (
    <div>
      <Greeting />  {/* Reuse here! */}
      <Greeting />  {/* And here!  */}
    </div>
  );
}
```

07 JSX – JavaScript XML

The syntax that makes React so expressive

JADE GLOBAL

What is JSX?

JSX lets you write HTML-like code inside JavaScript. It looks like HTML but compiles to React.createElement() calls.

JSX is NOT mandatory, but it makes React code much more readable.

JSX Rules

- Return one parent element
- Use className (not class)
- Use htmlFor (not for)
- Self-close empty tags:
- Expressions go in { curly braces }

JSX vs HTML

```
HTML:  class='box'
JSX:   className='box'

HTML:  onclick='fn()'
JSX:   onClick={fn}

HTML:  <img>
JSX:   <img />
```

JSX in Action — Expressions

JSX

```
const name = "React";
const age = 11;
return (
  <h1>Hello {name}! You are {age * 2} years old.</h1>
);
```

Ternary in JSX

JSX

```
const isLoggedIn = true;
return (
  <p>{isLoggedIn ? "Welcome back!" : "Please log in"}</p>
);
```

08 Props – Passing Data

Parent → Child communication in React

🔑 **Analogy:** Props are like function arguments — you pass data INTO a component the same way you pass arguments to a function.

<App />
Parent

passes props ↓

<UserCard />
Child

Passing Props (Parent)

JSX

```jsx
// In App.jsx (Parent)
function App() {
  return (
    <UserCard
      name="Alice"
      role="Developer"
    />
  );
}
```

Receiving Props (Child)

JSX

```jsx
// UserCard.jsx (Child)
function UserCard({ name, role }) {
  // Destructured props!
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}
```

Default Props

JSX

```jsx
// Set defaults so the component works even if a prop isn't passed
function UserCard({ name = "Guest", role = "Visitor" }) { ... }
```

✏️ **Mini Exercise:**

Create a <ProductCard /> component that accepts name, price, and description props. Render 3 different products in <App />.

09 State & useState Hook

Making components dynamic and interactive

JADE GLOBAL

State = Data that CAN CHANGE over time

When state changes ⟶ React automatically re-renders that component

Props = Data passed IN (read-only)

State = Data managed INSIDE the component (can change)

Classic Counter Example

JSX

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  const inc = () => setCount(count + 1);
  const dec = () => setCount(count - 1);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={inc}>+1</button>
      <button onClick={dec}>-1</button>
    </div>
  );
}
```

How Re-rendering Works

1. User clicks +1 button
2. setCount(count + 1) is called
3. React updates state value
4. Component re-renders with new count

⚠️ **NEVER directly mutate state: count++ ❌ Always use the setter: setCount(count + 1) ✅**

10 Event Handling

Responding to user interactions in React

JADE GLOBAL

onClick

Fires when element is clicked

JSX

```jsx
<button
  onClick={handleClick}>
  Click me
</button>
```

onChange

Fires when input value changes

JSX

```jsx
<input
  onChange={handleChange}
  value={text}
/>
```

onSubmit

Fires when form is submitted

JSX

```jsx
<form
  onSubmit={handleSubmit}>
  ...
</form>
```

Putting It Together — Event Handler Pattern

JSX

```jsx
function App() {
  const [text, setText] = useState("");

  const handleChange = (event) => {
    setText(event.target.value);  // event.target.value = what user typed
  };

  return <input onChange={handleChange} value={text} placeholder="Type here..." />;
}
```

11 Conditional Rendering

Show or hide UI based on state and conditions

if / else

JSX

```
function App() {
  const isLoggedIn = true;
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please log in.</h1>;
  }
}
```

Ternary Operator

JSX

```
// Best for inline conditional
return (
  <div>
    {isLoggedIn
      ? <h1>Welcome!</h1>
      : <h1>Log in</h1>}
  </div>
);
```

Logical AND (&&)

JSX

```
// Show ONLY if condition is true
return (
  <div>
    {hasError &&
      <p>Something went wrong!</p>}
  </div>
);
```

When to use each method:

`if / else`	When you need complex logic or multiple branches
`Ternary`	When you need either A or B — most common pattern
`&&`	When you only want to show something (or nothing)

12 Lists & Keys

Rendering dynamic lists of data in React

Rendering Lists

Use JavaScript's `.map()` to transform an array of data into an array of JSX elements.

🔑 **The key prop:**

React needs a unique key on each list item to track changes efficiently. Always use a unique identifier (like an ID), not the array index if possible.

⚠️ Avoid using index as key when items can be reordered or deleted — it causes subtle bugs.

Basic List Rendering

JSX

```
const fruits = ["Apple", "Banana", "Mango"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

12 Lists & Keys

Rendering dynamic lists of data in React

JADE GLOBAL

List with Objects (Real World)

JSX

```
const users = [
  { id: 1, name: "Alice", role: "Dev" },
  { id: 2, name: "Bob",   role: "Design" },
];

function UserList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name} –
{user.role}</li>
      ))}
    </ul>
  );
}
```

13 Forms & Controlled Components

Handling user input with React state

Controlled Components

In a controlled component, React state is the **single source of truth** for input values.

How it works:

1. Input value = React state

2. onChange → updates state

3. State drives input display

Result: You always know exactly what value is in the input — no need to query the DOM.

Complete Login Form Example

JSX

```jsx
import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState("");
  const [pass, setPass] = useState("");
  const onSubmit = (e) => {
    e.preventDefault();
    alert(`Email: ${email}`);
  };
  return (
    <form onSubmit={onSubmit}>
      <input type="email" value={email}
        placeholder="Your email"
        onChange={e => setEmail(e.target.value)}
      />
      <input type="password" value={pass}
        placeholder="Password"
        onChange={e => setPass(e.target.value)}
      />
      <button type="submit">Log In</button>
    </form>
  );
}
```

14 Styling in React

Multiple ways to make your components look great

JADE GLOBAL

CSS Files

JSX

```jsx
// App.css
.card {
  background: white;
  padding: 16px;
  border-radius: 8px;
}

// App.jsx
import './App.css';

function App() {
  return <div className="card">Hello</div>;
}
```

💡 Most common approach. Separate concerns — style in CSS, logic in JS.

Inline Styles

JSX

```jsx
// Inline styles use a JS object
// Properties are camelCase!
function App() {
  const style = {
    backgroundColor: 'blue',
    padding: '16px',
    borderRadius: '8px',
  };
  return <div style={style}>Hello</div>;
}
```

💡 Good for dynamic styles. Note: camelCase (backgroundColor not background-color)

CSS Modules

JSX

```jsx
// Button.module.css
.button {
  background: blue;
  color: white;
}

// Button.jsx
import styles from './Button.module.css';

function Button() {
  return <button className={styles.button}>
    Click
  </button>;
}
```

💡 Scoped styles — no class name collisions. Best for large apps.

15 useEffect – Side Effects

Running code in response to lifecycle events

JADE GLOBAL

What are Side Effects?

Side effects are anything that reaches OUTSIDE the component:

- Fetching data from an API
- Setting page title
- Setting up timers / intervals
- Subscribing to events

`useEffect(callback, [dependencies])`

Fetching Data on Mount

JSX

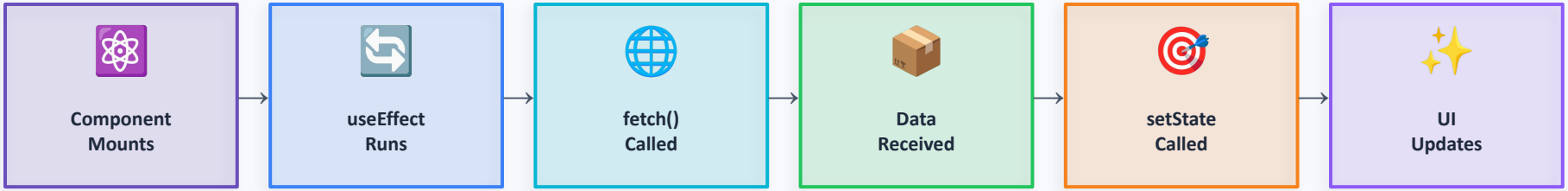
```jsx
import { useState, useEffect } from 'react';

function UserProfile() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // This runs ONCE when component mounts
    fetch('https://api.example.com/user/1')
      .then(res => res.json())
      .then(data => setUser(data));
  }, []); // Empty [] = run once

  if (!user) return <p>Loading...</p>;

  return <h1>Hello, {user.name}!</h1>;
}
```

Dependency Array Patterns

`useEffect(() => { ... })`	→ Runs after EVERY render
`useEffect(() => { ... }, [])`	→ Runs ONCE after first render (mount)
`useEffect(() => { ... }, [count])`	→ Runs when 'count' changes

16 Basic API Integration

Fetching and displaying real data in React



Complete API Fetch Pattern

JSX

```jsx
function PostList() {
  const [posts, setPosts]     = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(r => r.json())
      .then(data => { setPosts(data.slice(0,5)); setLoading(false); });
  }, []);

  if (loading) return <p>Loading...</p>;
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Best Practices

- ✅ Always handle loading state
- ✅ Always handle error state
- ✅ Use [] in useEffect dependency array
- ❌ Don't fetch inside render (causes loops)
- ❌ Don't forget to handle loading = false

17 React Router – Navigation

Client-side routing between pages without reloads

Install:

CMD
```
npm install react-router-dom
```

Core Router Building Blocks

`<BrowserRouter>`
Wraps your whole app. Enables routing.

`<Routes>`
Container for all Route entries.

`<Route path='/' element={<X/>}>`
Maps a URL to a component.

`<Link to='/about'>`
Navigate without page reload.

`useNavigate()`
Redirect programmatically.

Complete App.jsx Example

JSX
```jsx
import { BrowserRouter, Routes,
         Route, Link }
  from 'react-router-dom';

const Home  = () => <h1>Home Page</h1>;
const About = () => <h1>About Page</h1>;

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/"
          element={<Home />} />
        <Route path="/about"
          element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

18 Debugging React Apps

Finding and fixing bugs like a pro

JADE GLOBAL

console.log()

- Add inside functions and hooks
- Log state values to check updates
- Log props to verify what's passed
- console.log("state:", state);

JSX

```
function Counter() {
  const [count, setCount] = useState(0);
  console.log("Render! count =", count);
  ...
```

React DevTools

- Browser extension (Chrome / Firefox)
- See component tree visually
- Inspect props & state live
- Profile performance

JSX

```
Install: Chrome Web Store
→ "React Developer Tools"

Use: F12 → Components tab
```

Common Errors

- "each child should have a unique key" → add key prop
- "Cannot read properties of undefined" → check async data
- "Too many re-renders" → fix useEffect deps
- "is not a function" → check import names

JSX

```
// Fix: add key prop
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

19 Build & Deployment

Taking your React app from local to live on the internet

JADE GLOBAL

The Build Process

1

`npm run build`

Vite bundles your JS, CSS, and assets into optimized files in the /dist folder

→

2

`dist/ folder created`

Contains minified index.html, JS chunks, and CSS — ready to upload

→

3

`Upload to host`

Deploy the /dist folder to Netlify, Vercel, or GitHub Pages

→

4

`Live!` 🎉

Your app is live with a public URL anyone can visit

Free Hosting Options for Beginners

Netlify

EASIEST

1. npm run build
2. Drag /dist to netlify.com/drop
3. Get live URL instantly!

Vercel

BEST FOR TEAMS

1. Push code to GitHub
2. Connect repo at vercel.com
3. Auto-deploys on every push!

GitHub Pages

FREE FOREVER

1. Install gh-pages package
2. npm run deploy
3. username.github.io/repo

20 Common Beginner Mistakes

Avoid these pitfalls — every beginner faces them!

JADE GLOBAL

1	**Mutating state directly**	❌ `state.name = 'Alice'` ✅ `setState({ name: 'Alice' })`	*React won't detect the change or re-render*
2	**Missing key in lists**	❌ `{items.map(i => <li>{i}</li>)}` ✅ `{items.map(i => <li key={i.id}>{i.name}</li>)}`	*Causes performance bugs and wrong rendering*
3	**Forgetting () => in onClick**	❌ `<button onClick={setCount(1)}>` ✅ `<button onClick={() => setCount(1)}>`	*Calling function directly runs it on render!*
4	**Async in useEffect incorrectly**	❌ `useEffect(async () => { ... }, [])` ✅ `useEffect(() => { async function fn(){} fn(); }, [])`	*useEffect callback shouldn't be async directly*
5	**Missing e.preventDefault() in forms**	❌ `<form onSubmit={handleSubmit}>` ✅ `const handleSubmit = (e) => { e.preventDefault(); }`	*Without it, the page reloads on submit!*

🚀 MINI PROJECT

Build: Task Manager App

Apply everything you've learned in one real project

📝

Add Tasks

Controlled form with useState to add new tasks to the list

Skills: useState, Forms, onChange

✅

Complete Tasks

Toggle task as done/not done — update state in list

Skills: State updates, .map()

🗑️

Delete Tasks

Remove tasks by filtering state array on button click

Skills: Array.filter(), Keys

🔢

Task Counter

Show how many tasks are left dynamically

Skills: Derived state, Rendering

Tech Stack: React + useState + useEffect + React Router + fetch() + CSS Modules

🎉 You Did It!

JADE GLOBAL

You've completed the React.js Beginner Module. Here's what you now know:

- ⚛️ React Fundamentals
- 🧩 Components & JSX
- 📦 Props & State
- 🎯 Event Handling
- 🔄 useEffect & APIs
- 🗺️ React Router
- 🐛 Debugging Skills
- 🚀 Build & Deploy

What's Next? (Intermediate React)

Custom Hooks
Extract reusable logic from components

Context API
Share state across deeply nested components

Redux / Zustand
Advanced global state management

React Query
Powerful data fetching & caching

Next.js
Server-side rendering & full-stack React

Testing
Jest + React Testing Library